

Hands-on 4

Difference between JPA, Hibernate and Spring Data JPA

Aim

To understand the difference between JPA, Hibernate and Spring Data JPA and compare their implementation for performing database operations.

Java Persistence API (JPA)

Java Persistence API (JPA) is a Java specification used for persisting, reading and managing data between Java objects and relational databases.

Features

- JSR 338 Specification.
- Used for object-relational mapping (ORM).
- Provides standard APIs for database operations.
- Does not provide an implementation.
- Requires an implementation such as Hibernate.

Hibernate

Hibernate is an Object Relational Mapping (ORM) framework that implements the JPA specification.

Features

- Implements JPA.
- Maps Java objects to database tables.
- Automatically generates SQL queries.
- Supports CRUD operations.
- Supports caching and lazy loading.
- Reduces manual database coding.

Spring Data JPA

Spring Data JPA is a Spring framework module built on top of JPA.

Features

- Uses JPA implementation like Hibernate.
- Reduces boilerplate code.
- Provides built-in CRUD methods.
- Supports transaction management.
- Makes database programming easier.

Difference between JPA, Hibernate and Spring Data JPA

JPA	Hibernate	Spring Data JPA
JPA is a specification.	Hibernate is an implementation of JPA.	Spring Data JPA is a Spring module built on top of JPA.
Defines standard APIs.	Implements those APIs.	Simplifies database operations using repositories.
Cannot work independently.	Can work as a JPA provider.	Requires JPA implementation like Hibernate.
Database independent.	ORM Framework.	Provides Repository abstraction.
No direct implementation.	Complete ORM solution.	Reduces coding effort.

Hibernate Code Example

```
/* Method to CREATE an employee in the database */
public Integer addEmployee(Employee employee) {
    Session session = factory.openSession();
    Transaction tx = null;
    Integer employeeID = null;
    try {
        tx = session.beginTransaction();
        employeeID = (Integer) session.save(employee);
        tx.commit();
    }
    catch (HibernateException e) {
        if (tx != null)
            tx.rollback();
        e.printStackTrace();
    }
    finally {
        session.close();
    }
    return employeeID;
}
```

Explanation

- A Hibernate Session is opened.
- A transaction is started.
- The employee object is saved using session.save().
- The transaction is committed.
- If an exception occurs, the transaction is rolled back.
- Finally, the session is closed.

Spring Data JPA Code Example

EmployeeRepository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}
```

EmployeeService.java

```
@Autowired
private EmployeeRepository employeeRepository;

@Transactional
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}
```

Explanation

- EmployeeRepository extends JpaRepository.
- Spring automatically provides CRUD methods.
- No Session object is required.
- No Transaction object is required because @Transactional manages transactions.
- The employee is saved using save().

Comparison between Hibernate and Spring Data JPA

Hibernate	Spring Data JPA
Manual session handling.	Automatic session handling.
Manual transaction management.	Automatic transaction management using @Transactional.
Uses session.save().	Uses repository.save().
Requires more code.	Requires less code.
More complex.	Easier to use and maintain.

Advantages of JPA

- Standard persistence API.
- Database independent.
- Easy to switch between JPA implementations.
- Promotes portability.

Advantages of Hibernate

- Powerful ORM framework.
- Automatic SQL generation.
- Supports caching.
- Supports lazy loading.
- Reduces JDBC coding.

Advantages of Spring Data JPA

- Less boilerplate code.
- Built-in CRUD operations.
- Easy integration with Spring Boot.
- Automatic query generation.
- Better code maintainability.

Conclusion

JPA is a standard specification for object-relational mapping. Hibernate is an ORM framework that implements the JPA specification. Spring Data JPA is a Spring module that simplifies database operations by providing repository support and reducing boilerplate code. Among the three, Spring Data JPA offers the simplest and most efficient approach for developing database-driven Spring Boot applications.